

React JS

Title: React JS - Beginner to Advanced Notes

By: VK AI TECH by Kabir

1. What is React JS?

Definition

React JS is an **Open Source JavaScript Library** developed by **Meta (Facebook)** for building **fast, interactive and reusable User Interfaces (UI)**.

It follows a **Component-Based Architecture**, where every part of the UI is divided into reusable components.

Key Points

- JavaScript Library
 - Developed by Meta
 - Used for building UI
 - Component Based
 - Virtual DOM
 - Declarative Programming
-

2. Why do we need React JS for creating UI?

Traditional HTML + JavaScript becomes difficult when applications grow.

Examples

- Facebook
- Instagram
- Netflix
- WhatsApp Web
- Gmail

All these applications constantly update their UI.

React makes these updates:

- Faster
- Cleaner
- Easier
- Reusable

Advantages

- ✓ Reusable Components
 - ✓ Faster Rendering
 - ✓ Better Performance
 - ✓ Easy State Management
 - ✓ Large Community
 - ✓ Virtual DOM
 - ✓ Easy Maintenance
-

3. Drawbacks of HTML + JavaScript for creating UI

Without React

Developer needs to

- Find HTML element
- Remove DOM manually
- Update count manually
- Update cart manually
- Update price manually

Large code becomes difficult.

Problems

✗ Too much DOM manipulation

✗ Repeated code

✗ Difficult maintenance

✗ Slow UI

✗ Bugs increase

React solves this using Component Architecture.

4. What is React Element?

A React Element is a **JavaScript Object** that describes what should appear on the screen.

It is **NOT** a real HTML element.

React later converts it into DOM elements.

Example

```
const element =  
React.createElement(  
  "h1",  
  {},  
  "Hello"  
);  
  
React Element internally becomes  
{  
  type: "h1",  
  props: {},  
  children: "Hello"  
}
```

5. How to create React Element

A. Single Element

```
const heading =  
React.createElement(  
  "h1",  
  {},  
  "Welcome"  
);
```

B. Multiple Elements

```
const parent =  
React.createElement(  
  "div",  
  {},  
  [  
    React.createElement("h1", {}, "Heading"),  
    React.createElement("p", {}, "Paragraph")  
  ]  
);
```

Hierarchy

```
div  
|  
|--h1  
|  
|--p
```

6. What is ReactDOM?

ReactDOM is responsible for rendering React Elements into the Browser DOM.

Think of it as a bridge.

```
React Element  
  |  
  |  
ReactDOM  
  |  
  |  
Browser DOM
```

Example

```
const root =  
ReactDOM.createRoot(  
  document.getElementById("root")  
);  
  
root.render(element);
```

7. Why do we use JSX instead of `React.createElement()`?

Creating UI using `React.createElement()`

```
React.createElement(  
  "div",  
  {},  
  React.createElement(  
    "h1",  
    {},  
    "Hello"  
  )  
)
```

Looks difficult.

Now JSX

```
<div>  
  <h1>Hello</h1>  
</div>
```

Advantages

- ✓ Easy to Read
 - ✓ Easy to Write
 - ✓ Looks like HTML
 - ✓ Less Code
 - ✓ Better Maintenance
-

8. What is JSX?

- JSX means JavaScript XML
- It allows us to write HTML-like syntax inside JavaScript.
- Browser cannot understand JSX.
- It must be converted into JavaScript.
- Babel converts

```
<h1>Hello </h1>
```

into

```
React.createElement(  
  "h1",  
  {},  
  "Hello"  
)
```

JSX Rules

Rule 1

Return only one parent element



```
<div>
```

```
</div>
```



```
<h1></h1>
```

```
<p></p>
```

Rule 2

Close every tag

```
<img />
```

```
<input />
```

Rule 3

Use camelCase

```
onClick
```

```
className
```

```
htmlFor
```

Rule 4

JavaScript inside {}

```
<h1>{name}</h1>
```

Rule 5

Every JSX expression must return one element.

9. How to create JSX Elements

Single Element

```
const element = <h1>Hello React</h1>;
```

Nested Elements

```
const element = (  
  <div>  
    <h1>Hello</h1>  
    <p>React</p>  
  </div>  
)
```

10. Difference between HTML and JSX

Difference Between HTML and JSX

HTML

HTML is a markup language used to create web pages.

Uses `class` attribute.

Uses `for` attribute.

Uses lowercase event names like `onClick`.

Uses `tabindex` attribute.

Uses `readonly` attribute.

Uses `maxlength` attribute.

JSX

JSX is a JavaScript syntax used to create UI in React.

Uses `className` attribute.

Uses `htmlFor` attribute.

Uses camelCase event names like `onClick`.

Uses `tabIndex` attribute.

Uses `readOnly` attribute.

Uses `maxLength` attribute.

Boolean attributes can be written without a value (`checked`).

Boolean attributes are passed as JavaScript values (`checked={true}`).

Inline CSS is written as a string (`style="color:red"`).

Inline CSS is written as a JavaScript object (`style={{color: "red" }}`).

Cannot directly use JavaScript expressions.

Can use JavaScript expressions inside `{}`.

The browser understands HTML directly.

JSX must be converted into JavaScript using Babel before rendering.

Multiple root elements are allowed in an HTML document.

A component must return a single parent (root) element.

Some tags don't require self-closing.

Every tag must be properly closed (e.g., `<img />`).

Mainly used for static web page structure.

Mainly used for building dynamic and interactive React applications.

11. Vite Build Tool

Vite is a modern build tool used to create React applications.

Why Vite?

- ✓ Very Fast
 - ✓ Hot Module Replacement
 - ✓ Faster Development Server
 - ✓ Optimized Production Build
 - ✓ Supports TypeScript
 - ✓ Less Configuration
-

12. How to create React App?

Install

```
npm create vite@latest
```

```
Project Name  
  react-app
```

```
Select  
  React
```

```
Select
  JavaScript

Go inside
  cd react-app

Install packages
  npm install

Run project
  npm run dev
```

Why Build Tool?

A Build Tool

- Bundles files
 - Optimizes code
 - Converts JSX
 - Runs Development Server
 - Supports Modules
 - Handles Images
 - Hot Reload
-

13. Problems without Build Tool

Without Vite

Need to manually

- Import React CDN
- Import ReactDOM CDN
- Import Babel
- Configure Modules
- Configure Bundling
- Configure Minification

Problems

- ✗ Slow
 - ✗ Manual Configuration
 - ✗ No Hot Reload
 - ✗ Difficult Project Structure
-

14. React Folder Structure

```
project/  
  node_modules/  
  public/  
  src/  
    assets/  
    App.jsx  
    App.css  
    main.jsx  
    index.css  
  
package.json  
vite.config.js  
README.md
```

Explanation

node_modules

Downloaded Packages

public

Static Files

Logo

Images

favicon

src

Main Source Code

assets

Images

Icons

Videos

App.jsx

Main Component

main.jsx

Entry Point

package.json

Project Information

Dependencies

Scripts

vite.config.js

Vite Configuration

15. Props

1. Props means Properties
2. Props are used to send data

Example

```
<Greeting  
  name="Kabir"  
>
```

```
function Greeting(props){  
  
  return(  
    <h1>  
      {props.name}  
    </h1>  
  )  
  
}
```

Props are

- ✓ Read Only
 - ✓ Immutable
-

16. Functional Component

A Functional Component is a JavaScript Function that returns JSX.

Example

```
function Header(){  
  
  return(  
  
    <h1>  
    Welcome  
    </h1>  
  
  );  
  
}
```

Rules

- ✓ First Letter Capital
 - ✓ Return JSX
 - ✓ Reusable
-